7-Day DevOps Interview Master Plan

For a Professional with 4 Years of DevOps Experience

Use case: Production-grade three-tier application on AWS with React.js, Node.js, and PostgreSQL

Goal	**Become interview-ready in 7 intensive days with architecture, CI/CD, IaC, observability, security, and production operations depth.**
Workload	12 focused hours per day: concept revision, architecture design, hands-on practice, debugging, and mock interviews.
Target Stack	React.js frontend, Node.js API layer, PostgreSQL database, containerized deployment on AWS.
Outcome	You should be able to explain design decisions, implement a pipeline, troubleshoot failures, and discuss production SRE/DevOps practices confidently.

Prepared for structured interview preparation and practical implementation.

Table of Contents

1. Candidate Profile and Target Role

2. Production Architecture for Three-Tier Application

3. Production-Level DevOps Operating Model

4. Seven-Day Interview Plan (12 Hours per Day)

5. Hands-On Deliverables and Mock Interview Topics

6. Final Checklist for Interview Day

1. Candidate Profile and Target Role

This plan is designed for a DevOps engineer with around four years of experience who is expected to discuss both hands-on implementation and production decision making. The interviewer will usually check depth in Linux, networking, Git workflows, CI/CD, AWS services, containers, Infrastructure as Code, observability, incident handling, and security.

The strongest interview answers are not just tool names. They connect business requirements, architecture, deployment design, rollback strategy, cost, availability, and operational reliability.

Area	Expected Depth	Production Talking Points	Hands-on Focus
Linux & Networking	Strong	Process management, ports, DNS, TLS, reverse proxy, load balancing, troubleshooting	nginx, systemctl, netstat/ss, curl, dig
CI/CD	Strong	Branch strategy, build/test/package/deploy, approvals, rollback, artifact versioning	Jenkins/GitHub Actions/GitLab CI
AWS	Strong	VPC, ALB, Auto Scaling, ECS/EKS/EC2, RDS, IAM, CloudWatch, Secrets	Design and deployment
Containers & IaC	Strong	Docker image quality, Terraform modules, state, remote backend	Dockerfile, compose, terraform plan/apply
Observability & Ops	Medium to Strong	Logs, metrics, alerts, dashboards, runbooks, incident response	Prometheus/Grafana/ CloudWatch

2. Production Architecture for Three-Tier Application

The application consists of a React.js frontend, a Node.js backend API, and a PostgreSQL database. For interviews, the best production answer balances scalability, security, operability, and cost. A common and industry-standard deployment pattern on AWS is described below.

2.1 Recommended AWS Architecture

Users / Browsers
Amazon Route 53 -> AWS WAF -> Amazon CloudFront
Static React build hosted on Amazon S3 (or on Nginx container behind ALB when SSR is required)
Application Load Balancer (public subnets)
Node.js API service on Amazon ECS Fargate or EKS (private subnets, multiple AZs)
Amazon ElastiCache Redis (optional for cache/session/rate limit support)
Amazon RDS for PostgreSQL (private subnets, Multi-AZ, automated backups)
Amazon S3 for artifacts, backups, and logs archive
Amazon CloudWatch / Prometheus / Grafana / X-Ray / OpenTelemetry for observability
IAM, Secrets Manager, KMS, Security Groups, NAT Gateway, VPC endpoints, CI/CD pipeline

Preferred subnet layout: public subnets for ALB/NAT, private app subnets for containers, and isolated private DB subnets for PostgreSQL. This keeps the database inaccessible from the internet and restricts traffic through security groups and controlled application paths.

2.2 Component Responsibilities

Layer	AWS Services	Why Used	Production Notes
Frontend	S3 + CloudFront + Route 53 + ACM	Fast static delivery, HTTPS, CDN caching	Versioned build artifacts, cache invalidation, compression, security headers
API Layer	ALB + ECS Fargate/EKS + ECR	Scalable container runtime with rolling deployment	Health checks, auto scaling, blue/green or canary deployment
Database	RDS PostgreSQL	Managed relational database with backups and HA	Multi-AZ, parameter groups, PITR backups, read replicas when needed
Secrets	Secrets Manager / SSM Parameter Store	Avoid hard-coded credentials	Rotation, least privilege IAM, app injection at runtime
Logs & Metrics	CloudWatch, Prometheus, Grafana	Operational visibility	Error rate, p95 latency, CPU, memory, DB connections,

			disk

2.3 Request Flow

- User requests the application URL. Route 53 resolves the domain and CloudFront serves cached static content or forwards allowed paths.
- React.js assets are delivered from S3/CloudFront with HTTPS enforced through ACM certificates.
- API requests are sent to the Application Load Balancer, which forwards traffic to healthy Node.js containers.
- Node.js services read configuration and secrets securely, authenticate users, execute business logic, and connect to PostgreSQL.
- PostgreSQL stores transactional data. Optional Redis reduces repeated reads and supports session or queue-like patterns.
- Logs, metrics, traces, and alarms are emitted to monitoring platforms for proactive detection and troubleshooting.

2.4 High Availability, Scalability, and Security

Concern	Industry-Standard Practice	Interview Keywords
Availability	Multi-AZ deployment for ALB, ECS tasks, and RDS Multi-AZ	No single point of failure
Scalability	Horizontal scaling on app tier; CDN for frontend; read replicas and caching when needed	Auto Scaling, target tracking
Security	WAF, IAM least privilege, private subnets, security groups, KMS encryption, secret rotation	Defense in depth
Deployment Safety	Blue/green or rolling release with health checks and rollback	Zero-downtime deployment
Data Protection	Backups, PITR, snapshot policy, encryption at rest and in transit	RPO/RTO awareness

3. Production-Level DevOps Operating Model

Below is how work is normally done in production environments. Interviewers often expect not just deployment knowledge, but the full delivery lifecycle.

3.1 Source Control and Branching

- Use Git with a clear branching strategy such as trunk-based development or a lightweight GitFlow model.
- Protect main branches with pull requests, reviews, status checks, and signed or traceable commits when required.
- Every merge should produce a uniquely versioned artifact to guarantee traceability.

3.2 CI Pipeline

- Install dependencies, run linting, execute unit tests, perform security scans, and build immutable artifacts.
- Build React static assets and package Node.js service as a Docker image.
- Push container images to Amazon ECR with semantic version or commit SHA tagging.

3.3 CD Pipeline

- Deploy infrastructure changes through Terraform after plan approval.
- Deploy application changes to ECS/EKS using rolling, canary, or blue/green strategies.
- Run post-deployment smoke tests and trigger automatic rollback if health checks fail.

3.4 Infrastructure as Code

- Provision VPC, subnets, IAM, ALB, ECS/EKS, ECR, RDS, CloudWatch, and Secrets Manager using Terraform.
- Store state remotely in S3 with DynamoDB locking for team-safe operations.
- Design reusable modules and environment separation for dev, staging, and production.

3.5 Observability and SRE Practices

- Centralize logs, metrics, traces, dashboards, and alerts.
- Track golden signals: latency, traffic, errors, and saturation.
- Define runbooks, on-call ownership, severity levels, and escalation paths.

3.6 Security and Compliance

- Use image scanning, dependency scanning, IAM least privilege, WAF, patching, secret rotation, and audit logging.
- Keep SSH exposure minimal; prefer Session Manager or controlled bastion access where EC2 is used.
- Encrypt traffic with TLS and data with KMS-managed keys.

3.7 Sample Production CI/CD Flow

Step	Action	Tooling	Gate	Output
1	Developer raises PR	GitHub/GitLab/ Bitbucket	Code review	Merge candidate
2	CI validation	Node, npm, Jest, ESLint, SAST	Tests & quality pass	Build-ready code
3	Container build	Docker + ECR	Image scan	Immutable image
4	Infra validation	Terraform fmt/validate/plan	Approval	Safe execution plan
5	Staging deployment	ECS/EKS	Smoke tests	Release validation
6	Production deployment	Rolling/blue-green	Manual or policy approval	Live release
7	Post-release verification	CloudWatch/Grafana	SLO/health checks	Stable service

4. Seven-Day Interview Plan (12 Hours per Day)

The schedule below is intentionally intense. Each day is split into learning, hands-on work, revision, and speaking practice so that theory and practical memory reinforce each other.

Day 1 - Linux, Networking, Git, and Core DevOps Fundamentals

Time Block	Focus
Hour 1-2	Review Linux process management, services, permissions, logs, package management, SSH, and shell scripting.
Hour 3-4	Practice networking: DNS lookup, ports, TLS handshake basics, curl, traceroute, ss/netstat, nginx reverse proxy.
Hour 5-6	Revise Git branching, rebasing, merge conflict handling, pull request best practices, semantic versioning.
Hour 7-8	Hands-on: build a Linux VM/container lab; configure nginx to proxy /api to a Node.js app.
Hour 9-10	Write shell scripts for health check, log cleanup, and service restart automation.
Hour 11	Interview drill: explain CI vs CD, immutable artifacts, idempotency, and why infrastructure as code matters.
Hour 12	Revision notes and command cheat sheet creation.

Deliverables: one-page Linux command sheet, nginx reverse proxy lab, shell scripts, 20 interview questions answered aloud.

Day 2 - Docker, Containers, and Application Packaging

Time Block	Focus
Hour 1-2	Understand container lifecycle, images vs containers, union filesystem, networking, volumes, and registries.
Hour 3-4	Write optimized Dockerfiles for React build and Node.js API; learn multistage builds.
Hour 5-6	Hands-on: dockerize React app, Node.js app, and local PostgreSQL with docker-compose.
Hour 7-8	Practice health checks, environment variables, secrets injection patterns, and image tagging strategy.
Hour 9-10	Debug common failures: container crash loop, port mismatch, missing env var, dependency issues.
Hour 11	Explain when to use ECS, EKS, or EC2-based deployment.
Hour 12	Summarize best practices: small images, non-root user, deterministic builds, vulnerability scans.

Deliverables: production-style Dockerfile, docker-compose stack, troubleshooting notes, spoken comparison of ECS vs EKS.

Day 3 - AWS Architecture, VPC, Compute, Load Balancing, and Storage

Time Block	Focus
Hour 1-2	Revise AWS global infrastructure, IAM, VPC, subnets, route tables, internet/NAT gateways, and security groups.
Hour 3-4	Design the three-tier architecture on paper: Route 53, CloudFront, S3, ALB, ECS/EKS, RDS PostgreSQL.
Hour 5-6	Hands-on: create Terraform skeleton for VPC, subnets, security groups, and ALB.
Hour 7-8	Review RDS PostgreSQL configuration: Multi-AZ, backups, parameter groups, storage, connections, and failover.
Hour 9-10	Learn scaling patterns: ECS service auto scaling, stateless APIs, cache layer, read replicas.
Hour 11	Interview drill: explain public vs private subnets and how traffic reaches the database securely.
Hour 12	Redraw architecture from memory and speak through request flow end to end.

Deliverables: architecture diagram, Terraform VPC scaffold, RDS notes, traffic flow explanation.

Day 4 - CI/CD, Jenkins or GitHub Actions, Artifact Management, and Rollback

Time Block	Focus
Hour 1-2	Review pipeline stages: checkout, lint, test, build, scan, package, push, deploy, verify.
Hour 3-4	Hands-on: create pipeline for Node.js and React applications.
Hour 5-6	Add Docker image build and push to registry, then deployment manifest or ECS task definition update.
Hour 7-8	Integrate Terraform plan stage with approval gate before apply.
Hour 9-10	Practice blue/green, rolling deployment, canary release, and rollback decision logic.
Hour 11	Mock explanation: how to prevent a bad deployment from impacting customers.
Hour 12	Capture pipeline templates and deployment checklist.

Deliverables: working CI/CD pipeline YAML or Jenkinsfile, rollback checklist, release strategy comparison.

Day 5 - Terraform, Environment Strategy, Secrets, and Security

Time Block	Focus
Hour 1-2	Revise Terraform basics: providers, resources, variables, outputs, modules, workspaces, backend.
Hour 3-4	Hands-on: create reusable modules for VPC, ECS service, RDS, and security groups.
Hour 5-6	Configure remote backend in S3 with DynamoDB locking; discuss state safety and team workflows.
Hour 7-8	Implement secrets strategy using Secrets Manager or SSM Parameter Store.
Hour 9-10	Review IAM least privilege, KMS encryption, WAF basics, image scanning, and dependency scanning.
Hour 11	Interview drill: explain why hard-coded secrets and manual console changes are dangerous.
Hour 12	Refactor code and prepare neat Terraform directory structure.

Deliverables: Terraform module layout, remote backend config, secrets approach, IAM/security talking points.

Day 6 - Monitoring, Logging, Incident Handling, and Performance

Time Block	Focus
Hour 1-2	Revise logs, metrics, traces, dashboards, alerts, SLI/SLO, and the four golden signals.
Hour 3-4	Hands-on: instrument Node.js app logs and health endpoints; define metrics to collect.
Hour 5-6	Create CloudWatch/Grafana dashboard plan for CPU, memory, p95 latency, error rate, request volume, DB health.
Hour 7-8	Simulate incidents: high CPU, database connection exhaustion, failed deployment, disk alerts, 5xx spikes.
Hour 9-10	Practice root cause analysis and write mini runbooks for top five incidents.
Hour 11	Mock explanation: how to monitor success after a production deployment.
Hour 12	Review backup, restore, and disaster recovery expectations for PostgreSQL.

Deliverables: dashboard design, alert list, five runbooks, incident response answers.

Day 7 - Full Mock Interview Day and End-to-End System Design

Time Block	Focus
Hour 1-2	Draw the complete AWS production architecture without notes.
Hour 3-4	Explain end-to-end CI/CD flow, infra provisioning, deployment, observability, and rollback.
Hour 5-6	Solve scenario questions: traffic spike, DB outage, certificate expiry, failed release, cost optimization.
Hour 7-8	Perform hands-on recap: review Dockerfiles, Terraform modules, pipeline YAML, and monitoring plan.
Hour 9-10	Take a full mock interview: technical questions, troubleshooting questions, and architecture round.
Hour 11	Review weak areas and prepare crisp 2-minute answers for core concepts.
Hour 12	Rest the brain, finalize notes, and create interview-day cheat sheet.

Deliverables: final architecture explanation, mock interview answers, final cheat sheet, confidence in production-level discussion.

5. Hands-On Deliverables and Mock Interview Topics

By the end of the seven days, aim to have the following deliverables ready:

- A clean architecture diagram for the React + Node.js + PostgreSQL application on AWS.
- A production-style Dockerfile for the Node.js API and a static build deployment strategy for React.
- A docker-compose or local environment definition for development and testing.
- A Terraform repository or module skeleton for networking, compute, load balancing, and database provisioning.
- A CI/CD pipeline definition with tests, image build, registry push, deployment, and rollback notes.
- A monitoring checklist covering app, infrastructure, database, and release health.
- A set of runbooks for common incidents: 5xx errors, CPU spikes, DB connection issues, failed deploys, and secrets/config mistakes.

5.1 Important Mock Interview Questions

- How would you deploy a React frontend, Node.js backend, and PostgreSQL database securely on AWS?
- Why would you choose ECS Fargate over EKS, and when would EKS be justified?
- How do you implement zero-downtime deployments and reliable rollback?
- How do you manage secrets in production and keep them out of Git?
- What metrics and alerts would you define for this application?
- How do you troubleshoot high latency when the frontend is fine but API response time is degraded?

- What happens when the database becomes unreachable? How does the application fail gracefully?
- How would you separate dev, staging, and production environments in Terraform and CI/CD?
- How do you secure the database so it is not exposed publicly?
- How do you handle backups, restore validation, and disaster recovery for PostgreSQL?

5.2 Strong Answer Pattern

A strong answer usually follows this pattern: requirement -> design choice -> implementation -> risk -> monitoring -> rollback. This makes your response sound like a production engineer instead of a tool operator.

6. Final Checklist for Interview Day

Area	What to Review	Ready?
Architecture	Route 53, CloudFront, S3, ALB, ECS/EKS, RDS PostgreSQL, security groups, private subnets	☐
Containers	Dockerfile, compose, health checks, env vars, image optimization	☐
CI/CD	Pipeline stages, approvals, deployment strategy, rollback	☐
Terraform	Modules, backend, state locking, env separation	☐
Monitoring	Logs, metrics, alerts, dashboards, runbooks	☐
Security	IAM, secrets, WAF, KMS, TLS, patching, scans	☐
Troubleshooting	CPU, memory, DB connection, DNS, SSL, deployment failures	☐
Communication	2-minute crisp explanation for architecture and release process	☐

Final advice: speak in terms of reliability, repeatability, security, and business impact. Interviewers usually remember candidates who can explain why a design is safe in production, not just how to run a command.